

Chương 2:

CẤU TRÚC CỦA MỘT CHƯƠNG TRÌNH ANDROID

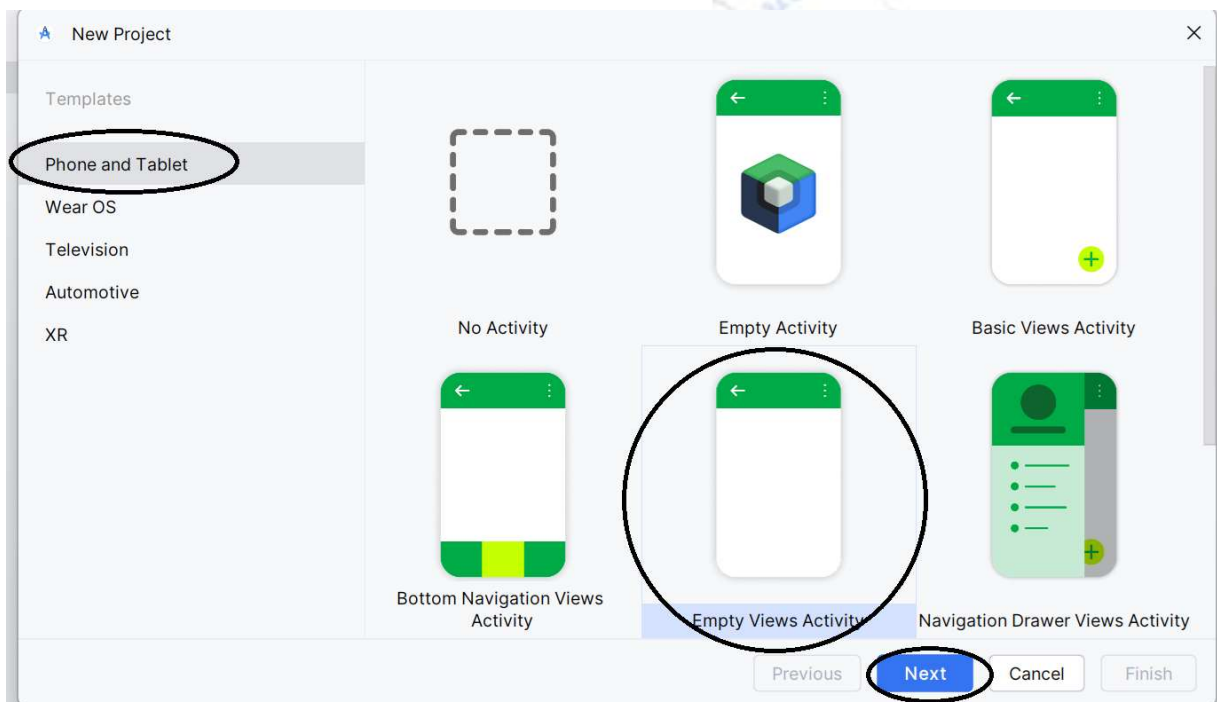
Mục tiêu của chương 2: Chương này sẽ giới thiệu đến các bạn cách viết chương trình Android đầu tiên và cách biên dịch nó. Chương này cũng sẽ giới thiệu đến các bạn các thành phần cơ bản của một ứng dụng Android và vòng đời của các thành phần đó.

2.1. Viết chương trình đầu tiên trên Android Studio

Viết chương trình hiển thị dòng chữ: “XIN CHAO MOI NGUOI, TOI LA KHUONG”.

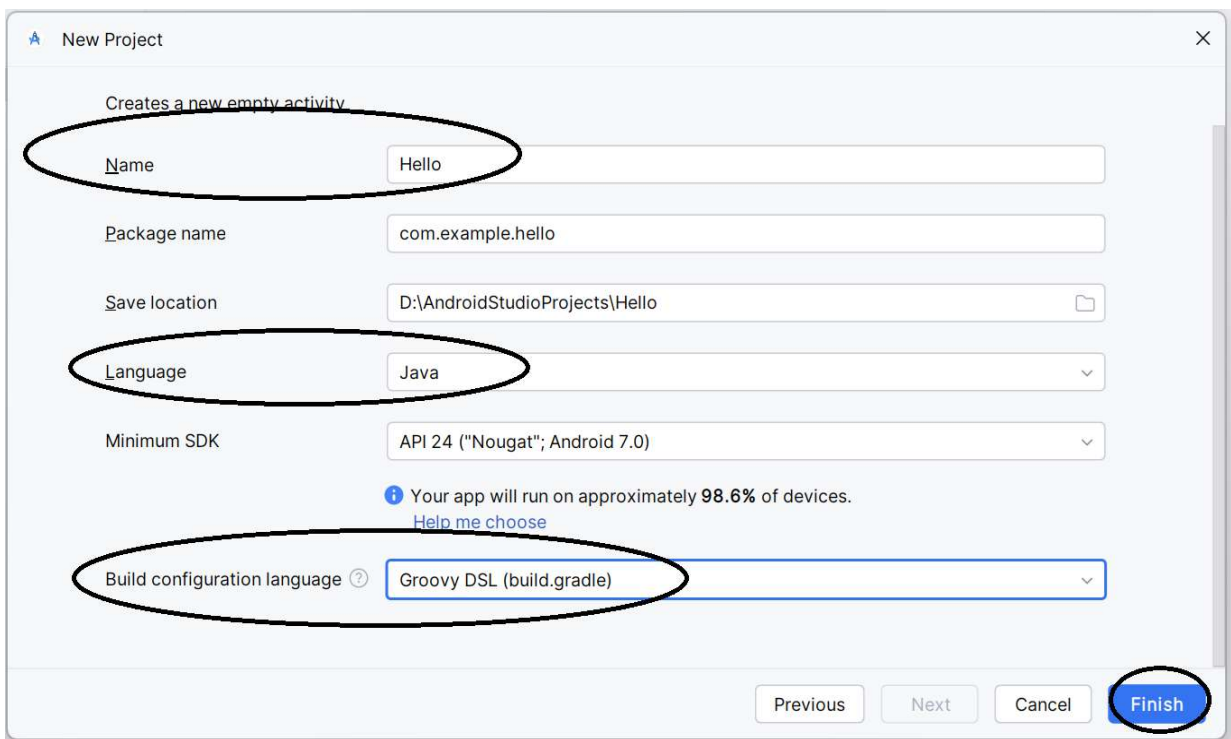
Bước 1: Tạo một dự án mới

- Chọn File / New / New Project. Xuất hiện menu, chọn “Phone and Tablet”, chọn “Empty View Activity”, chọn “Next”

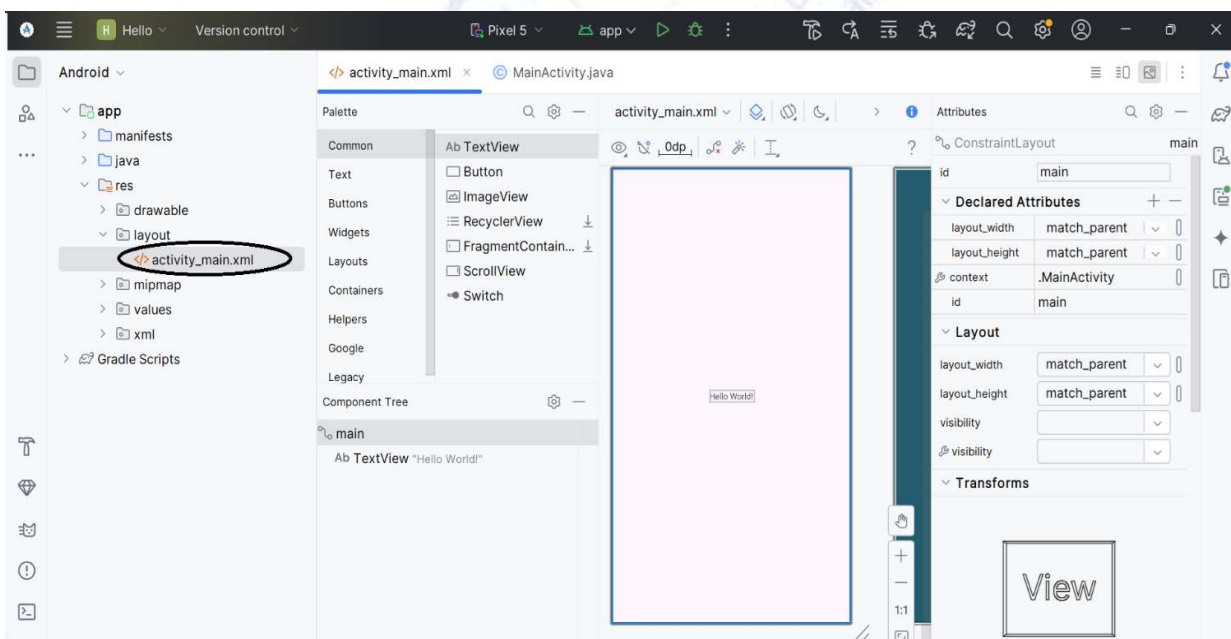


- Chọn các thông tin về Object (dự án), xong chọn “Finish”. Giải thích các mục thiết lập:

- Mục **Phone and Tablet**: ứng dụng sẽ được chạy trên điện thoại và máy tính bảng.
- Chọn tên dự án và đường dẫn lưu dự án.
- Chọn ngôn ngữ: Java
- Mục **Minimum SDK** : Chọn giới hạn API thấp nhất mà ứng dụng có thể cài đặt, nên chọn API thấp để phù hợp hơn cho các thiết bị cũ (ví dụ API 24)
- Chọn **Build configuration language**: Groovy DSL

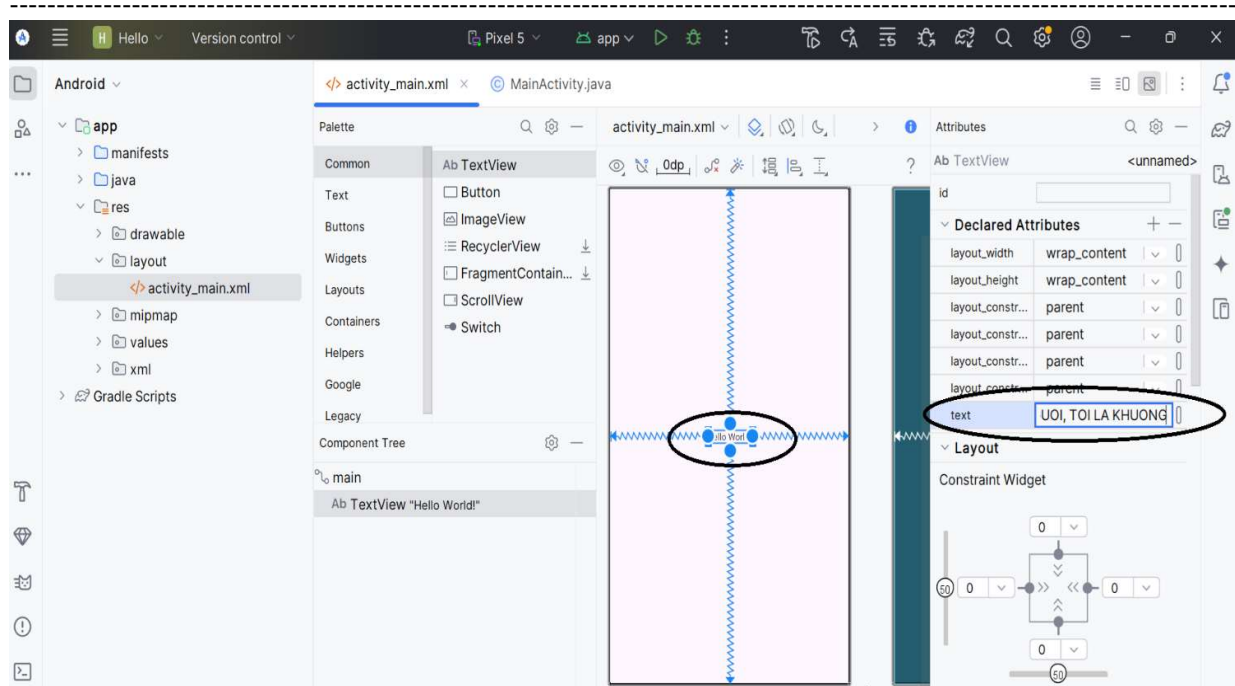


- Chờ một lát để dự án khởi tạo, xong click vào tập tin **activity_main.xml** sẽ thấy giao diện như bên dưới:

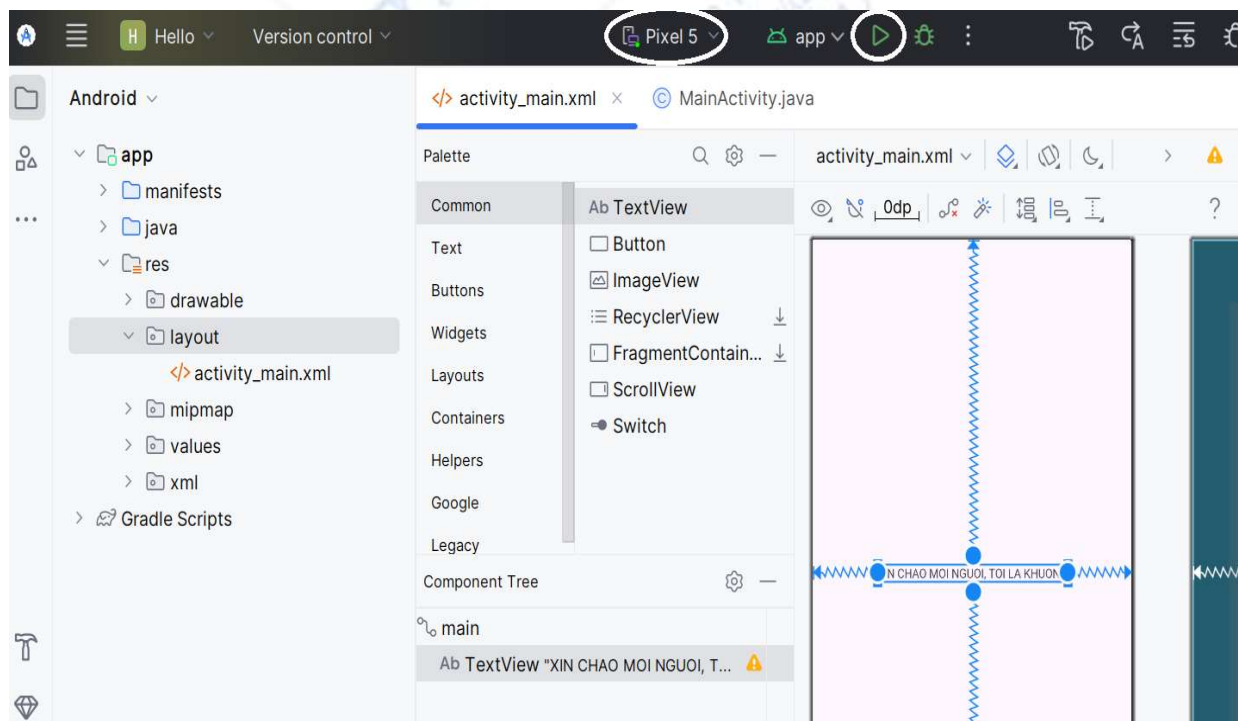


Bước 2: Chương trình hiện tại sẽ mật định hiển thị chữ “Hello world”, bây giờ chúng ta sẽ sửa lại để hiển thị theo yêu cầu là: “XIN CHAO MOI NGUOI, TOI LA KHUONG” bằng cách như sau:

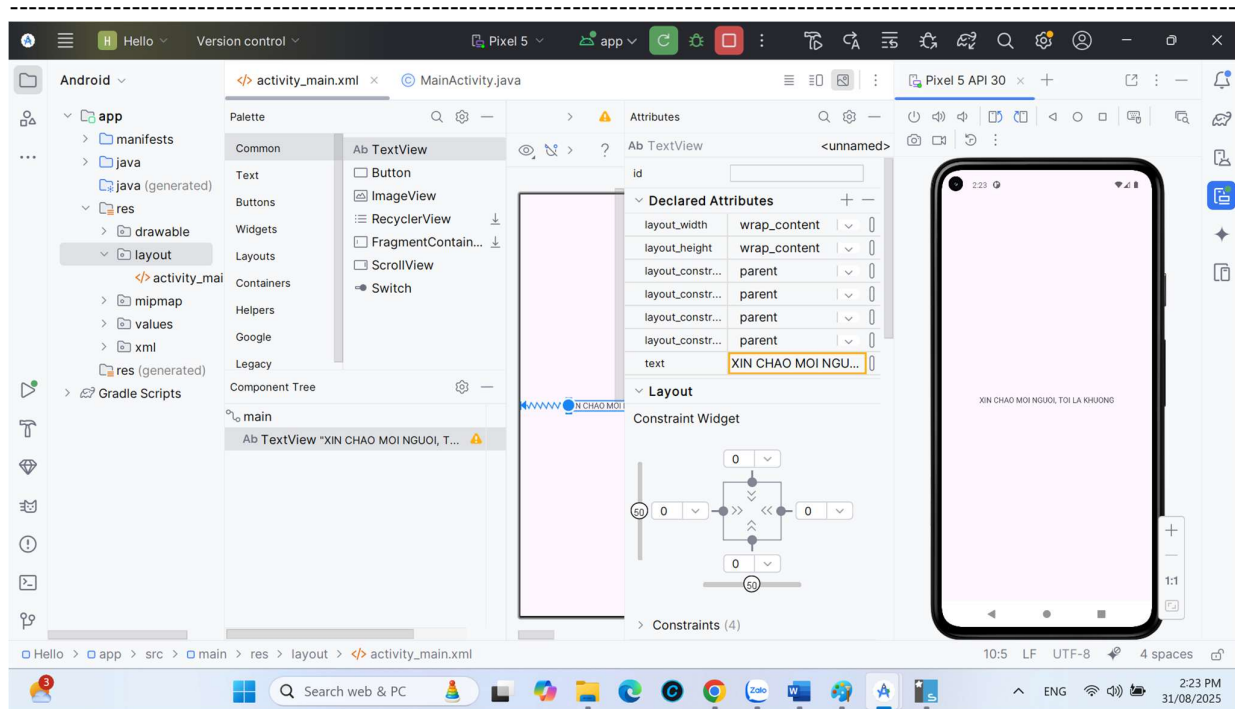
- Click vào Text: Hello world!, bên phải sẽ xuất hiện thuộc tính của Text, thay đổi nội dung thuộc tính text thành “XIN CHAO MOI NGUOI, TOI LA KHUONG”



- Thay thế giá trị “Hello world!” là: XIN CHAO MOI NGUOI, TOI LA KHUONG.
- Lưu dự án lại: chọn File / Save All.
- Chọn máy ảo để biên dịch chương trình, xong chọn biểu tượng chạy chương trình như bên dưới:

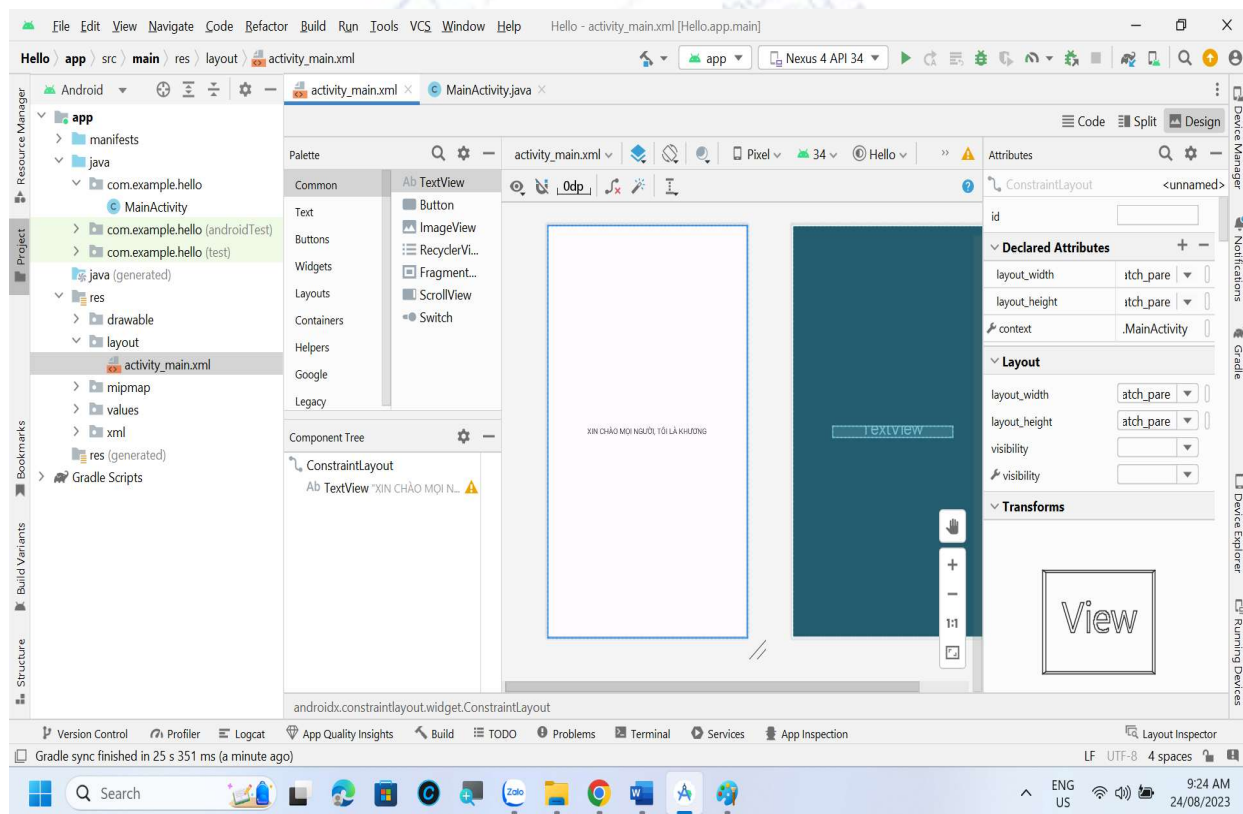


- Chờ một tí chương trình sẽ chạy ra như sau:



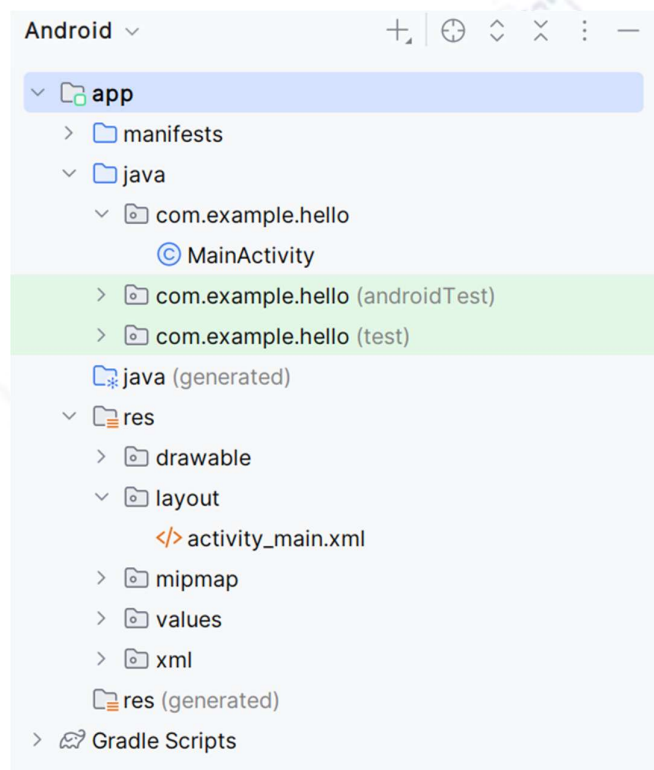
2.2. Cấu trúc thư mục và tập tin của một Project Android

Mở lại dự án **Hello** đã làm ở bài trước, ta sẽ thấy một dự án Android sẽ có các thành phần như sau:



2.2.1. Cấu trúc thư mục của Project Android

- Thư mục res/: chứa các tài nguyên của ứng dụng, như các icons và các tài nguyên đồ họa khác.
- Thư mục res/drawable/: chứa file hình ảnh (PNG, JPEG,...).
- Thư mục res/layout/: chứa các tập tin định dạng giao diện, dưới dạng XML.
- Thư mục res/menu/: chứa các tập tin chi tiết về các menu, dưới dạng XML.
- Thư mục res/values/: chứa các tập tin strings, dimensions,... dùng để định nghĩa các chuỗi xuất
- Thư mục java: chứa file xử lý chương trình .java



Hình 2.1: Cấu trúc thư mục của Project Android

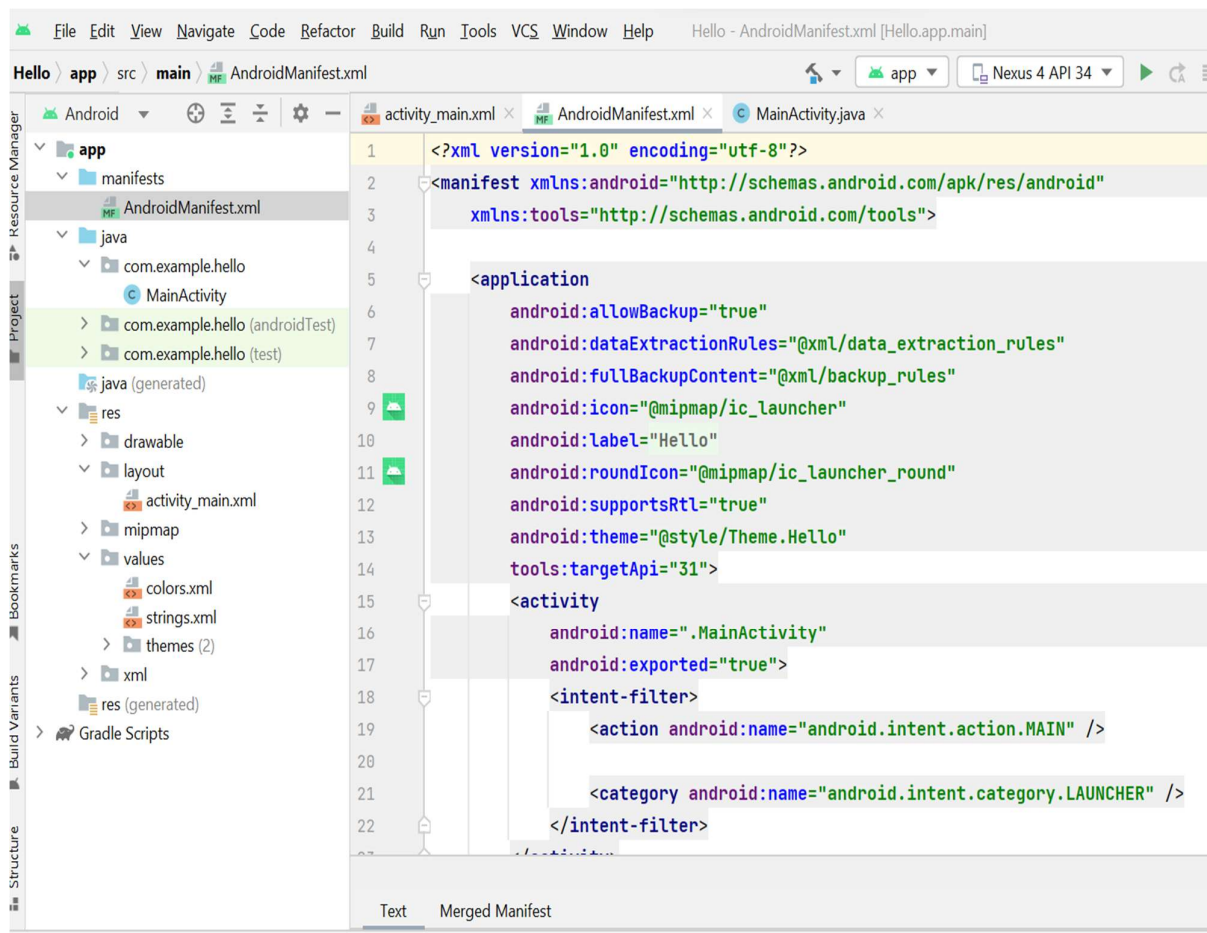
2.2.2. Giới thiệu các tập tin của Project Android

Hãy quan sát **MainActivity.java** và **activity_main.xml**. Khi một ứng dụng được tạo ra thì thông thường sẽ có một Activity để khởi chạy ứng dụng. Ở đây bạn hiểu rằng MainActivity.java chính là class chứa toàn bộ source code, còn activity_main.xml chính là phần giao diện. Đối với Android khi một Activity tạo ra thì thường nó đi kèm với một Layout giao diện nào đó (Tức là nó luôn được tách thành 2 phần: phần source code riêng và phần giao diện riêng). Bạn hiểu Activity giống như là các màn hình (cửa sổ) tương tự như là C#, mỗi Activity là một màn hình tương tác cụ thể nào đó.

Bạn cũng cần nhớ rằng bất kỳ một Activity nào muốn được triệu gọi thành công trong Android Project thì bắt buộc nó phải được khai báo trong tập tin **AndroidManifest.xml**. Nếu như bạn gọi một Activity mà Activity này không được khai báo trong Manifest thì chắc chắn chương trình sẽ bị lỗi và tắt luôn.

- **Tập tin AndroidManifest.xml:**

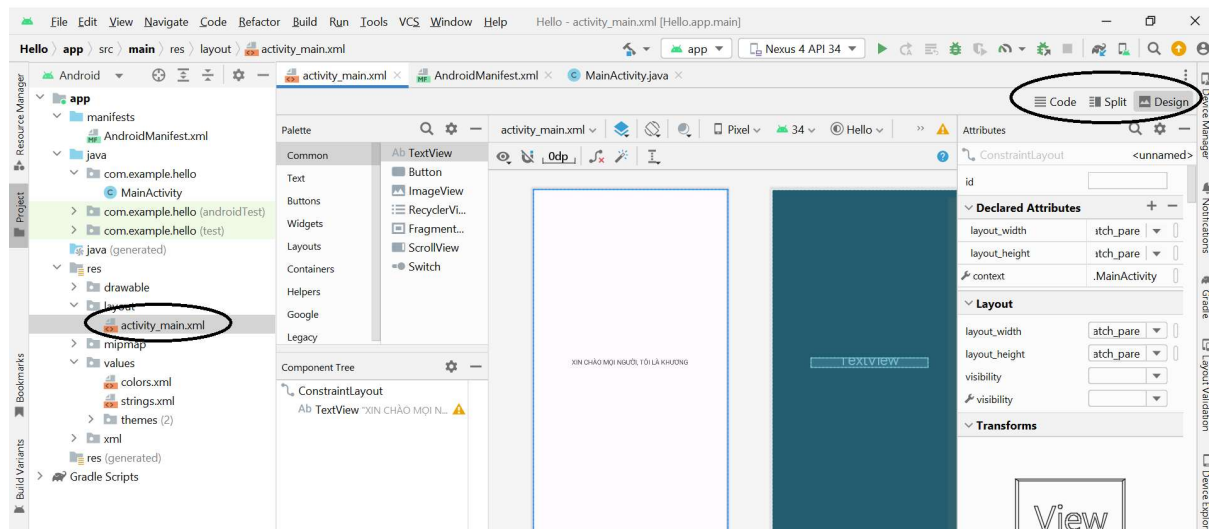
Hãy xem cấu trúc bên trong của AndroidManifest.xml (double click vào nó và chọn tab như hình bên dưới):



- Như hình bên trên bạn thấy đó, MainActivity muốn được triệu gọi thì nó phải được khai báo trong này. Và đặc biệt nếu như muốn nó là màn hình đầu tiên được thực thi khi chạy ứng dụng thì bạn phải khai báo giống như tag `<intent-filter>` ở trên. Như vậy nếu bạn muốn một Activity bất kỳ nào đó được gọi đầu tiên khi chạy ứng dụng thì bạn chỉ khai báo y chang như vậy, còn các Activity khác bạn không cần khai báo `<intent-filter>` như trên (tức là hoàn toàn không có tag này)

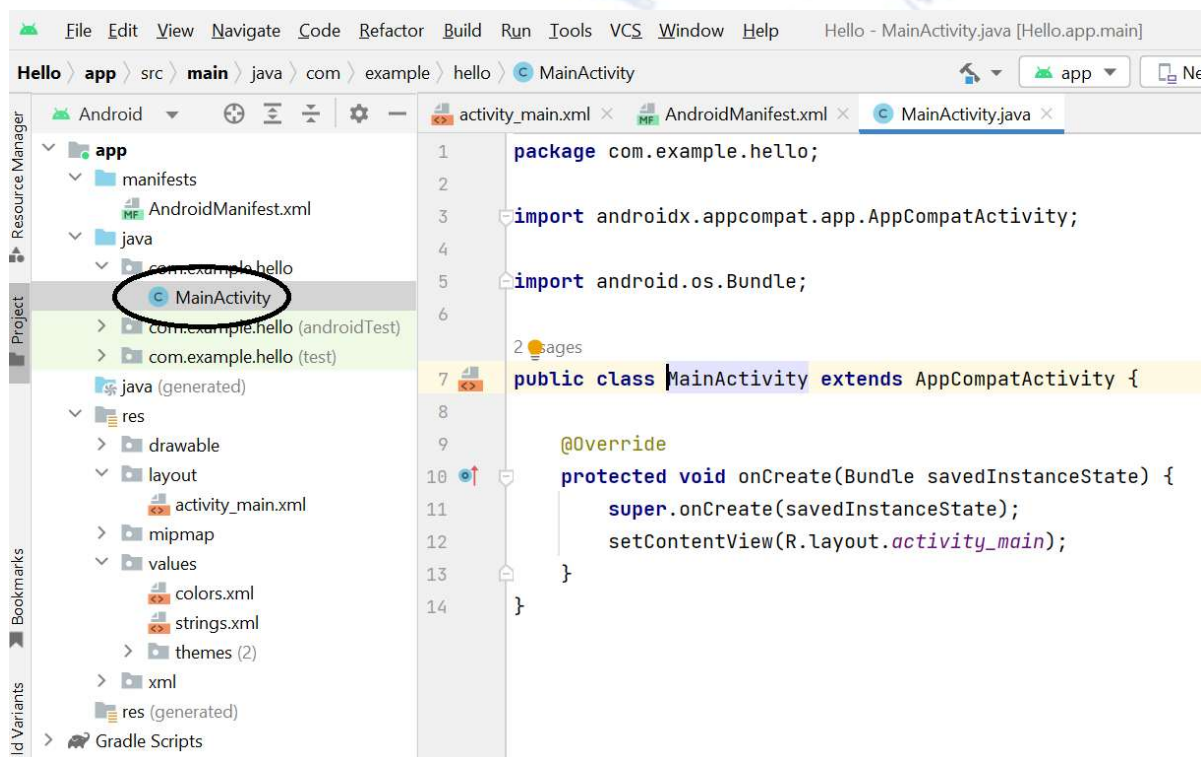
- **Tập tin activity_main.xml:**

- Tiếp theo bạn double – click vào activity_main.xml, tập tin này cho phép người dùng thiết kế giao diện của chương trình bằng cách trực quan hay viết code thông qua việc chọn nút “Code”, “Split” hay “Design”



- Tập tin MainActivity.java:

- Tiếp tục double – click vào MainActivity.java. Tập tin này chứa tất cả các mã lệnh để chương trình hoạt động.



- Trong màn hình trên, bạn thấy bên trong hàm **onCreate** có lệnh:

+ `setContentView(R.layout.activity_main)` ==> thiết lập giao diện cho Activity.
Với `activity_main` lấy từ R.java

+ `findViewById(R.id.btnXinChao)` ==> truy suất control là Button trên giao diện.
với `btnXinChao` lấy từ R.java

2.3. Các thành phần của một ứng dụng Android



Hình 2.2: Các thành phần chính của một ứng dụng Android

2.3.1. Activity (giao diện hoạt động)

Một **Activity** đại diện cho một giao diện người sử dụng trực quan, ở đó người dùng có thể thực hiện những gì họ muốn làm. Ví dụ một activity thể hiện một màn hình ở đó có danh bạ của người sử dụng, người sử dụng có thể lựa chọn danh bạ hiển thị số điện thoại hoặc hiển thị ảnh và chú thích của từng contact. Một chương trình nhắn tin SMS có thể bao gồm các activity là: một activity thể hiện một danh sách các số liên lạc (có thể gửi được tin nhắn), một activity thể hiện một trang để người dùng viết tin nhắn và lựa chọn gửi đến một tài khoản nào đó, và một activity khác lại cho phép xem lại các tin đã gửi hoặc chỉnh sửa chúng. Mặc dù tất cả các activity kết hợp lại mới tạo nên một ứng dụng có giao diện cho người sử dụng, nhưng mỗi activity vẫn hoạt động độc lập với những cái còn lại. Mỗi một activity là một lớp con của lớp cơ sở Activity.

Một ứng dụng có thể bao gồm một hoặc nhiều activity. Và đương nhiên có bao nhiêu activity, mỗi activity hoạt động ra sao, giao diện trông thế nào là do ứng dụng quyết định và thiết kế. Thông thường thì có một activity được đánh dấu là activity đầu tiên, và người dùng có thể thấy nó ngay khi ứng dụng được khởi chạy. Mỗi một activity có một cửa sổ làm việc để sử dụng và thao tác trên đó. Thông thường mỗi cửa sổ này đều lấp đầy màn hình. Nhưng cũng có khi nó nhỏ hơn và nổi ngay trên một cửa sổ làm việc khác.

Những nội dung trực quan ở mỗi cửa sổ làm việc được cung cấp bởi một hệ thống các view, nó là những đối tượng được sử dụng để tạo nên giao diện người dùng và được thừa kế từ lớp cơ sở View. Mỗi một view điều khiển một khu vực có dạng hình chữ nhật đặc biệt bên trong cửa sổ. Mỗi một lớp cha, parent views, bao gồm và tổ chức cách sắp xếp các lớp con của nó. Những view là lá (không có con) hiển thị trên khu vực mà chúng điều khiển và trả lời các hành động do người dùng tác động vào khu vực nó kiểm soát. Do đó có thể nói, mỗi một view là nơi mà activity tương tác với người sử dụng. Ví dụ, một view có thể hiển thị một ảnh mà khi người sử dụng click vào cái ảnh đó sẽ có một hành động được thực thi.

Android có sẵn một số view cho chúng ta sử dụng như là button (nút), text field (trường văn bản), scroll bar (thanh cuộn), menu (thanh thực đơn), check box (hộp chọn), ...

Một view được đặt vào bên trong 1 cửa sổ của activity bởi phương thức Activity setContentView(). Đối tượng ContentView là đối tượng gốc của cây phân cấp các view.

2.3.2. Service (dịch vụ)

Một service thì không có một giao diện trực quan, nhưng nó có thể chạy ngầm định trên thiết bị và không giới hạn thời gian hoạt động. Ví dụ như một service giúp thiết bị có thể chơi nhạc ngoài desktop trong khi người sử dụng đang làm công việc khác, hay nó có thể tải dữ liệu trên mạng về máy, hoặc tính toán cái gì đó và sẵn sàng cung cấp kết quả khi có một activity cần đến nó. Mỗi một service được thừa kế extend từ lớp cơ sở Service.

Ví dụ đơn giản nhất là trình chơi nhạc từ một play list (danh sách bài hát). Trình chơi nhạc chắc chắn có một hoặc nhiều hơn một activity có khả năng để người sử dụng chọn bài hát và bắt đầu chơi chúng. Tuy nhiên người sử dụng mong muốn khi thoát khỏi trình chơi nhạc, nhạc vẫn được chơi bình thường, và activity thì không có khả năng thực hiện điều đó vì nó không giao tiếp với người dùng nữa. Để giữ cho nhạc vẫn tiếp tục được chơi thì activity của trình chơi nhạc gọi một service đến và chạy nó ở dạng ngầm định. Hệ thống sẽ tiếp tục chạy service này ngay cả khi activity thoát ra ngoài (không còn trên màn hình nữa).

Chúng ta hoàn toàn có thể kết nối tới một service đang chạy hoặc nếu chúng chưa bắt đầu chạy thì có thể khiến chúng chạy. Khi đã kết nối với service, bạn có thể giao tiếp với nó thông qua một giao diện mà service hiển thị.

Ví dụ với service chơi nhạc thì người sử dụng có thể play, stop, pause, next, rewind hoặc chơi lại từ đầu.

Giống như activity và các thành phần (component) khác, service chạy trên luồng (thread) xử lý chính của ứng dụng. Do đó chúng không thể đóng các thành phần khác hay can thiệp vào giao diện của người dùng. Chúng thường sinh ra 1 luồng mới cho các tác vụ bị chi phối thời gian thực hiện. Chi tiết về thread chúng ta sẽ khảo sát kỹ hơn trong các phần sau.

2.3.3. Broadcast receivers (bộ thu phát)

Một bộ broadcast receiver là một thành phần không làm gì cả nhưng lại nhận và tương tác với các thông báo được đưa ra.

Nhiều bộ phát (broadcast) hình thành trong mã của hệ điều hành – ví dụ, thông báo rằng timezone (múi giờ) đã được thay đổi, rằng pin yếu, hoặc thông báo user vừa đổi ngôn ngữ hiển thị. Một ứng dụng cũng có thể bắt đầu phát thông báo – ví dụ, để cho các ứng dụng khác biết dữ liệu trên mạng đã được tải về thiết bị và sẵn sàng sử dụng. Một ứng dụng cũng có thể không có bất kỳ bộ thu phát nào nếu chúng không cần trả lời các thông báo từ thiết bị hay ứng dụng khác.

Tất cả các bộ thu phát thì kế thừa từ lớp cơ sở BroadcastReceiver. Các bộ thu phát thì không có giao diện sử dụng, tuy nhiên chúng có thể khởi động một ứng dụng khi cần phản hồi các thông tin mà chúng nhận được. Hoặc chúng có thể sử dụng đối tượng NotificationManager để cảnh báo cho người dùng biết.

Các thông báo có thể gây chú ý tới người dùng theo những cách khác nhau – ví dụ như nháy đèn màn hình, rung, chơi nhạc,... Thông thường thì chúng tạo ra một thông báo “cứng” trên màn hình hiện thời của thiết bị, để người dùng có thể thấy và đọc những thông báo đó.

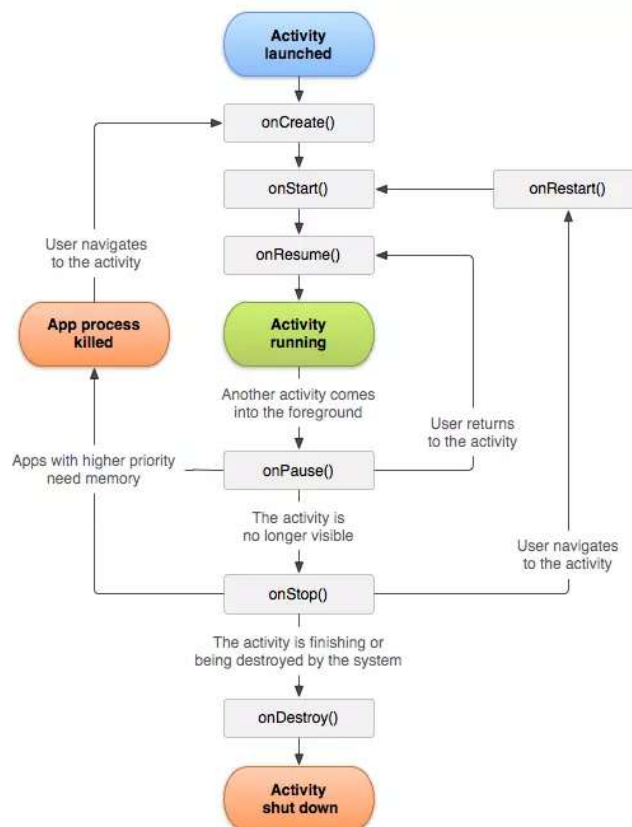
2.3.4. Content providers (Bộ cung cấp nội dung)

Một content provider tạo ra một tập hợp các dữ liệu đặc biệt của ứng dụng này mà các ứng dụng khác có thể dùng chung. Dữ liệu này có thể được lưu ở file hệ thống, trong cơ sở dữ liệu SQLite hoặc trong bất cứ chỗ nào lưu trữ được trên thiết bị. Và cũng như những thành phần khác, content provider được thừa kế từ lớp cơ sở ContentProvider, hiện thực các phương thức mặc định, cho phép các ứng dụng khác có thể gọi nó và lưu trữ/ truy cập đến dữ liệu của nó. Tuy nhiên các ứng dụng không thể gọi trực tiếp các phương thức. Chúng gọi các phương thức này gián tiếp qua việc gọi một đối tượng, đó là ContentResolver. Một ContentResolver có thể giao tiếp với bất kỳ content provider nào, chúng dựa vào nhau để quản lý các tiến trình xử lý thông tin phức tạp.

2.4. Vòng đời của các thành phần ứng dụng Android

Các thành phần ứng dụng đều có vòng đời của nó – đó là sự bắt đầu khi Android khởi động thành phần cho đến khi kết thúc ứng dụng, thực thể ứng dụng bị hủy. Các trạng thái ứng dụng có thể có hiệu lực hay không có hiệu lực; đối với activities, thì có tình trạng visible hoặc invisible. Trong mục này, chúng ta khảo sát về vòng đời của activities, services, và broadcast receiver.

2.4.1. Vòng đời của Activity



Hình 2.3: Vòng đời của một Activity

Một **Activity** có ba trạng thái:

- Đang hiệu lực hoặc đang chạy: khi nó trực tiếp giao tác với người dùng.
- Tạm dừng: khi nó không còn được hiệu lực nhưng vẫn được nhìn thấy bởi người dùng. Nói dễ hiểu hơn, có Activity này nằm trên một Activity khác mà Activity này trong suốt hoặc không che phủ hết màn hình vì vậy Activity khác được nhìn thấy. Một Activity đang dừng thì hoàn toàn “còn sống” nhưng có thể bị “giết đi” bởi hệ thống trong trường hợp bộ nhớ thấp.
- Dừng hẳn: nếu Activity bị che khóa hoàn toàn bởi một Activity khác. Nó vẫn còn duy trì tất cả trạng thái và thông tin thành viên. Tuy nhiên, người dùng không còn thấy nó vì màn hình của nó ẩn đi và thường bị “giết” bởi hệ thống khi không đủ bộ nhớ.

Nếu một Activity bị tạm dừng và dừng hẳn, hệ thống có thể loại bỏ nó ra khỏi bộ nhớ bằng cách gọi hàm `finish()` hoặc loại bỏ nó ra khỏi tiến trình. Khi nó thể hiện trước người dùng lần nữa, nó phải khởi động và khôi phục lại các trạng thái trước đó.

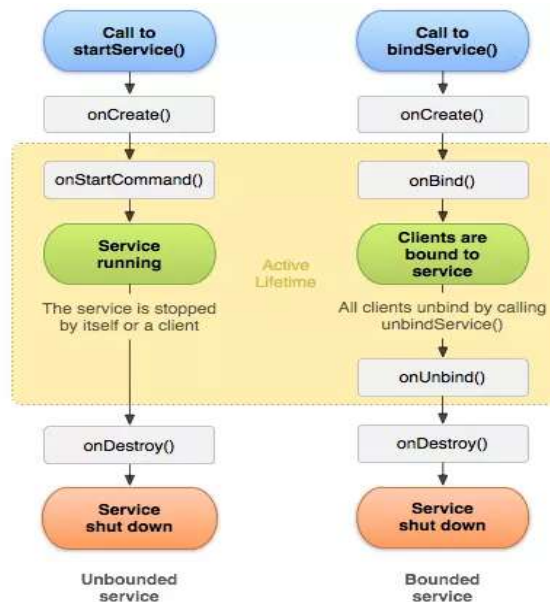
Nếu chú ý, ta thấy có những vòng khép kín như sau:

- Vòng khép kín toàn bộ của ứng dụng: Activity bắt đầu từ lần gọi hàm đầu tiên `onCreate()` đến hàm cuối cùng `onDestroy()`. Một Activity làm tất cả các thiết lập đầu tiên cho chương trình trong hàm `onCreate()` và giải phóng tất cả các tài nguyên trong hàm `onDestroy()`. Ví dụ như ứng dụng có 1 luồng chạy ngầm để tải dữ liệu từ máy chủ về thì nó tạo luồng trong hàm `onCreate()` và sau đó dừng hẳn luồng trong hàm `onDestroy()`.
- Vòng đời nhìn thấy được bởi người dùng: trong Activity từ hàm `onStart()` cho đến hàm `onStop()`. Trong suốt thời gian này, người dùng có thể thấy Activity trên màn hình. Giữa 2 phương thức này, bạn có thể duy trì các tài nguyên mà cần thiết để trình bày Activity cho người dùng. Ví dụ, bạn có thể khai báo `BroadcastReceiver` trong hàm `onStart()` để giám sát sự thay đổi ảnh hưởng đến giao diện và giải phóng nó trong hàm `onStop()` khi người dùng không còn nhìn thấy những gì bạn đang trình bày. Hàm `onStart()` và `onStop()` được gọi nhiều lần bởi vì activity thay đổi nhiều lần giữa trạng thái nhìn thấy được và ẩn đi đối với người dùng.
- Vòng đời chạy ngầm: từ hàm `onResume()` đến hàm `onPause()`. Trong suốt thời gian này, Activity chuyển từ trạng thái `resume` sang `pause` hay ngược lại. `onPause()` được gọi khi thiết bị “ngủ” hoặc khi Activity mới được khởi động, hàm `onResume()` được gọi khi kết quả của một Activity hay một intent mới được đưa ra. Vì thế mã lệnh trong hai phương thức này nên ít thôi.

Hình trên mô tả những vòng khép kín và con đường mà activity phải trải qua giữa các trạng thái. Hình ô van thể hiện trạng thái của Activity, hình chữ nhật thể hiện các phương thức của vòng đời ứng dụng khi mà Activity có sự thay đổi trạng thái.

- Phối hợp các activities: khi một Activity khởi động một Activity khác, cả hai Activity sẽ thay đổi trạng thái của vòng đời của mình. Activity này tạm dừng và có thể dừng hẳn trong khi cái khác thì mới khởi động. Trong trường hợp bạn có thể cần phối hợp các Activity này khi mà 2 Activity cùng nằm trong cùng một tiến trình.

2.4.2. Vòng đời của Service



Hình 2.4: Vòng đời của Service

Một service có thể dùng trong 2 cách:

- Nó có thể được khởi động và được cho phép dùng cho đến khi ai đó dừng hẳn nó hoặc nó tự dừng hẳn. Để khởi động một service, ta dùng hàm `Context.startService()` và dừng hẳn bằng hàm `Context.stopService()` hoặc tự dừng bằng hàm `Service.stopSelf()` hoặc `Service.stopSelfResult()`. Chú ý là hàm `stopService()` chỉ được gọi một lần duy nhất để dừng service trong khi đó gọi hàm `startService()` bao nhiêu lần tùy thích.

- Nó có thể được vận hành bằng mã lệnh dùng lớp interface mà nó đã định nghĩa. Client thiết lập kết nối và dùng kết nối ấy để gọi dịch vụ. Kết nối được thiết lập bằng cách gọi `Context.bindService()` và được đóng bởi hàm `Context.unbindService()`.

Giống như Activity, service có những vòng đời của riêng mình và có các phương thức thực thi để giám sát sự thay đổi trong trạng thái của nó. Nhưng service có ít hơn và gồm có 3 phương thức public: `void onCreate()`, `void onStart(Intent intent)`, `void onDestroy()`.

Bằng cách thực thi những phương thức này, chúng ta có thể giám sát vòng đời của service:

- Toàn bộ vòng đời của service xảy ra khi gọi hàm `onCreate()` và kết thúc khi gọi hàm `onDestroy()`. Giống như một Activity, một service khởi động trong hàm `onCreate()` và giải phóng mọi tài nguyên trong hàm `onDestroy()`.

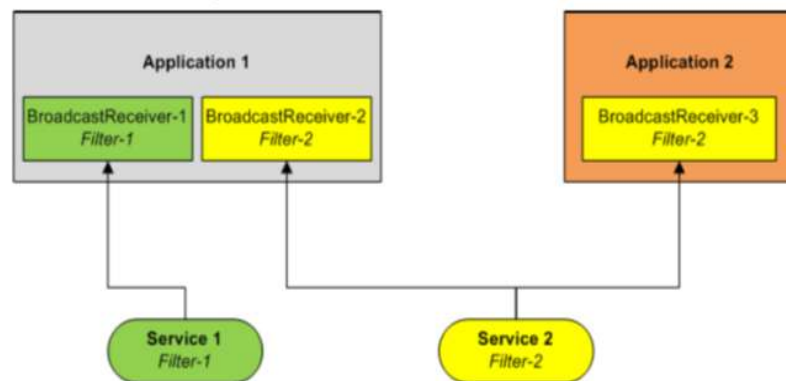
- Vòng đời “có hiệu lực” của service bắt đầu khi gọi hàm `onStart()`, nhưng không được kết thúc bằng `onStop()` vì hàm này không có thực.

Phương thức `onCreate()` và `onDestroy()` được gọi cho tất cả dịch vụ cho dù service khởi động bằng hàm `Context.startService()` hay hàm `Context.bindService()`. Tuy nhiên,

onStart() được gọi khi service đã khởi động bằng hàm startService(). Nếu một service cho phép những service khác liên kết với nó thì phải dùng các phương thức sau: onBind(Intent intent), boolean onUnbind(Intent intent), void onRebind(Intent intent).

Hình sau mô tả các phương thức của service. Mặc dù hình sau chia ra 2 loại service dựa trên tiêu chí khởi động bởi hàm startService () và hàm bindService(). Cho dù nó khởi động bằng cách nào đi chăng nữa, thì nó vẫn cho phép client kết nối tới nó và bất kỳ service nào cũng có thể gọi 2 hàm onBind() và onUnbind().

2.4.3. Vòng đời BroadcastReceiver



Hình 2.5: Vòng đời BroadcastReceiver

Broadcast receiver có một hàm cho phép người dùng viết lại là hàm void onCreate(Context curContext, Intent broadcastMsg).

Khi một tin nhắn phát đi cho người nhận, Android gọi phương thức onReceive và chuyển nó cho đối tượng Intent đang chứa tin nhắn. Broadcast receiver được xem là có hiệu lực chỉ khi nào nó đang thực thi phương thức này. Khi phương thức onReceive() kết thúc thì broadcast receiver không còn hiệu lực nữa.

Một tiến trình với một broadcast receiver thì được bảo vệ khỏi bị “giết”. Nhưng một tiến trình với các thành phần không còn hiệu lực (Activity, service,...) thì bị hệ thống “giết” bất kỳ lúc nào khi mà bộ nhớ không đủ cho các tiến trình khác.

2.4.4. Tiến trình và vòng đời ứng dụng

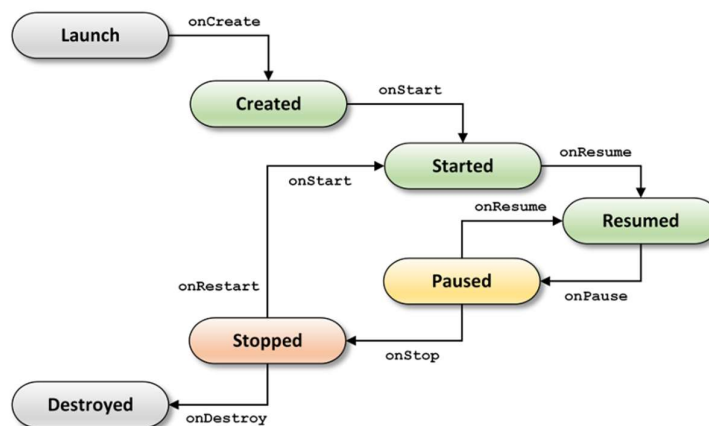
Hệ thống Android duy trì một tiến trình ứng dụng càng lâu càng tốt nhưng nó sẽ loại bỏ các tiến trình cũ khi mà bộ nhớ thấp. Để quyết định tiến trình nào được giữ lại, tiến trình nào bị “giết” đi, Android đặt mỗi tiến trình vào một “hệ thống cấp bậc quan trọng” dựa trên các thành phần đang chạy trên nó và trạng thái của các thành phần đó.

Những tiến trình với độ quan trọng thấp bị loại bỏ đi đầu tiên, sau đó tới lượt các tiến trình có độ ưu tiên cao hơn và cứ như vậy. Có 5 cấp trong hệ thống cấp bậc gồm các cấp bậc sau:

- Tiến trình nền: là tiến trình người dùng đang làm việc. Một tiến trình được xem là tiến trình nền nếu có một trong các điều kiện sau đây:

- + Nó đang chạy một activity mà người dùng đang tương tác.
- + Có đối tượng Service đang thực thi một trong những phương thức onCreate(), onStart(), onDestroy().
- + Có đối tượng BroadcastReceiver mà đang thực thi hàm onReceive().
- Tiến trình nhìn thấy được: là tiến trình không có các thành phần nền nhưng vẫn có thể ảnh hưởng đến những gì mà người dùng thấy trên màn hình. Một tiến trình được xem là thấy nếu thỏa cả hai điều kiện sau:
 - + Nó giữ một activity mà không phải activity nền nhưng vẫn nhìn thấy được đối với người dùng. Điều này xảy ra khi Activity hiện tại là một hộp thoại và Activity trước đó có thể nhìn thấy được do hộp thoại không che hết màn hình.
 - + Nó giữ một service mà đang kết nối tới một Activity nhìn thấy được.
- Tiến trình dịch vụ: là tiến trình chạy một service được bắt đầu bởi hàm startService() mà nó không rơi vào 2 trạng thái trên. Mặc dù tiến trình dịch vụ không liên quan gì tới bất kỳ cái gì mà người dùng thấy, nhưng nó làm những thứ mà người dùng quan tâm đến chẳng hạn như đang chơi nhạc mp3 ngầm, và vì thế hệ thống giữ cho chúng chạy nếu đủ bộ nhớ để duy trì 2 tiến trình trên.
- Tiến trình ngầm: là tiến trình giữ một Activity mà người dùng không nhìn thấy được. Những tiến trình này không ảnh hưởng trực tiếp lên người dùng, nó có thể bị “giết” bất kỳ lúc nào để dành bộ nhớ cho 3 dạng tiến trình trên.
- Tiến trình rỗng: là tiến trình không giữ bất cứ một ứng dụng nào cả. Chỉ có một lý do để giữ những tiến trình như vậy là xem nó như là một vùng đệm để cải thiện tốc độ, thời gian khởi động vào lần tới khi mà các thành phần cần nó để chạy nhanh hơn. Hệ thống thường “giết” các tiến trình này để cân bằng tài nguyên hệ thống giữa vùng đệm tiến trình và vùng đệm lỗi bên dưới thiết bị.

Android xếp hạng các tiến trình từ cao đến thấp dựa trên mức độ quan trọng của các thành phần hiện đang có hiệu lực trong tiến trình.



Hình 2.6: Tiến trình và vòng đời ứng dụng Android

2.5. Câu hỏi ôn tập chương 2

Câu 2.1: Hãy nêu các thư mục và tập tin quan trọng trong một dự án Android

Câu 2.2: Hãy nêu các thành phần của một ứng dụng Android

Câu 2.3: Hãy nêu các vòng đời của một ứng dụng Android? Vì sao lập trình viên cần hiểu rõ vòng đời Activity?

Câu 2.4: Ý nghĩa của các phương thức onCreate(), onStart(), onResume().

Câu 2.5: Điều gì xảy ra khi hệ thống Android cần giải phóng bộ nhớ?

ThS Nguyễn Văn Khương
Giảng viên Tin học, ĐH Quang Nam
Email: khuongdx@gmail.com
Mobile: 0905.065317